

# 100 SQL Commands Cheatsheet

 **100 commands, organized by what you're trying to do — not alphabetically.** From basic SELECT to transaction control, this is the full surface of SQL you'll actually touch.

## What's Covered

| #  | Section                                                                                                               | Commands |
|----|-----------------------------------------------------------------------------------------------------------------------|----------|
| 1  |  Data Definition (DDL)               | #1–10    |
| 2  |  Data Manipulation (DML)             | #11–20   |
| 3  |  Querying & Filtering                | #21–30   |
| 4  |  Joins                              | #31–38   |
| 5  |  Aggregation & Grouping            | #39–48   |
| 6  |  Window Functions                  | #49–58   |
| 7  |  Subqueries & CTEs                 | #59–65   |
| 8  |  String Functions                  | #66–75   |
| 9  |  Date & Time Functions             | #76–83   |
| 10 |  Constraints, Transactions & Admin | #84–100  |

## Data Definition (DDL) — #1 to #10

1. `CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50));`
2. `ALTER TABLE employees ADD COLUMN salary DECIMAL(10,2);`
3. `ALTER TABLE employees DROP COLUMN salary;`
4. `ALTER TABLE employees RENAME COLUMN name TO full_name;`
5. `DROP TABLE employees;`
6. `TRUNCATE TABLE employees; -- removes all rows, keeps structure`
7. `CREATE INDEX idx_name ON employees(name);`
8. `DROP INDEX idx_name;`

```
9. CREATE SCHEMA sales;
10. RENAME TABLE employees TO staff;
```

## ● Data Manipulation (DML) — #11 to #20

```
11. INSERT INTO employees (id, name) VALUES (1, 'Priya');
12. INSERT INTO employees VALUES (2, 'Sam'), (3, 'Jae'); -
    - multi-row insert
13. UPDATE employees SET salary = 75000 WHERE id = 1;
14. DELETE FROM employees WHERE id = 3;
15. INSERT INTO archive SELECT * FROM employees WHERE terminated = TRUE;
16. UPDATE employees SET salary = salary * 1.1; -- update every row
17. DELETE FROM employees; -- deletes all rows, keeps table
18. INSERT INTO employees (id, name) VALUES (1, 'Priya') ON
    CONFLICT (id) DO NOTHING;
19. UPDATE employees SET manager_id = NULL WHERE manager_id = 5;
20. MERGE INTO target USING source ON target.id = source.id
    WHEN MATCHED THEN UPDATE SET target.val = source.val
    WHEN NOT MATCHED THEN INSERT (id, val) VALUES (source.id, source.val);
```

## ● Querying & Filtering — #21 to #30

```
21. SELECT * FROM employees;
22. SELECT DISTINCT department FROM employees;
23. SELECT name FROM employees WHERE salary > 70000;
24. SELECT * FROM employees WHERE department IN ('Eng', 'Sales');
25. SELECT * FROM employees WHERE hire_date BETWEEN '2024-01-01' AND '2024-12-31';
26. SELECT * FROM employees WHERE name LIKE 'A%';
27. SELECT * FROM employees WHERE manager_id IS NULL;
```

```
28. SELECT * FROM employees ORDER BY salary DESC LIMIT 10;
29. SELECT * FROM employees ORDER BY salary DESC OFFSET 10
LIMIT 10; -- pagination
30. SELECT name AS employee_name FROM employees;
```

## ● Joins — #31 to #38

```
31. SELECT * FROM a INNER JOIN b ON a.id = b.a_id;
32. SELECT * FROM a LEFT JOIN b ON a.id = b.a_id;
33. SELECT * FROM a RIGHT JOIN b ON a.id = b.a_id;
34. SELECT * FROM a FULL OUTER JOIN b ON a.id = b.a_id;
35. SELECT * FROM employees e1 JOIN employees e2 ON e1.manager_id = e2.id; -- self join
36. SELECT * FROM sizes CROSS JOIN colors;
37. SELECT * FROM a LEFT JOIN b ON a.id=b.a_id WHERE b.a_id IS NULL; -- anti-join
38. SELECT * FROM a NATURAL JOIN b; -- joins on all same-named columns
```

## ● Aggregation & Grouping — #39 to #48

```
39. SELECT COUNT(*) FROM employees;
40. SELECT department, SUM(salary) FROM employees GROUP BY department;
41. SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) > 60000;
42. SELECT department, COUNT(DISTINCT manager_id) FROM employees GROUP BY department;
43. SELECT MIN(salary), MAX(salary) FROM employees;
44. SELECT department, SUM(CASE WHEN active THEN 1 ELSE 0 END) AS active_count FROM employees GROUP BY department;
45. SELECT department, salary, GROUPING(department) FROM employees GROUP BY ROLLUP(department, salary);
46. SELECT department, COUNT(*) FROM employees GROUP BY department ORDER BY COUNT(*) DESC;
47. SELECT STDDEV(salary), VARIANCE(salary) FROM employees;
```

```
48. SELECT department, STRING_AGG(name, ', ') FROM employee
s GROUP BY department;
```

## Window Functions — #49 to #58

```
49. SELECT name, RANK() OVER (ORDER BY salary DESC) FROM em
ployees;
50. SELECT name, DENSE_RANK() OVER (ORDER BY salary DESC) F
ROM employees;
51. SELECT name, ROW_NUMBER() OVER (PARTITION BY department
ORDER BY salary DESC) FROM employees;
52. SELECT name, NTILE(4) OVER (ORDER BY salary DESC) FROM
employees;
53. SELECT date, SUM(amount) OVER (ORDER BY date) FROM orde
rs;
54. SELECT date, AVG(amount) OVER (ORDER BY date ROWS BETWE
EN 6 PRECEDING AND CURRENT ROW) FROM orders;
55. SELECT month, revenue, LAG(revenue) OVER (ORDER BY mont
h) FROM sales;
56. SELECT month, revenue, LEAD(revenue) OVER (ORDER BY mon
th) FROM sales;
57. SELECT name, FIRST_VALUE(salary) OVER (PARTITION BY dep
artment ORDER BY salary DESC) FROM employees;
58. SELECT name, PERCENT_RANK() OVER (ORDER BY salary) FROM
employees;
```

## Subqueries & CTEs — #59 to #65

```
59. SELECT name FROM employees WHERE salary > (SELECT AVG(s
alary) FROM employees);
60. SELECT name FROM employees e WHERE EXISTS (SELECT 1 FRO
M orders o WHERE o.emp_id = e.id);
61. SELECT * FROM (SELECT *, RANK() OVER (ORDER BY salary D
ESC) AS rnk FROM employees) t WHERE rnk <= 3;
62. WITH high_earners AS (SELECT * FROM employees WHERE sal
ary > 100000) SELECT * FROM high_earners;
```

```

63. WITH RECURSIVE org AS (
    SELECT id, name, manager_id FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, e.name, e.manager_id FROM employees e JOIN
    org o ON e.manager_id = o.id
) SELECT * FROM org;
64. SELECT name FROM employees WHERE id NOT IN (SELECT emp_id FROM orders);
65. SELECT name, (SELECT COUNT(*) FROM orders o WHERE o.emp_id = e.id) AS order_count FROM employees e;

```

## String Functions — #66 to #75

```

66. SELECT UPPER(name), LOWER(name) FROM employees;
67. SELECT TRIM(name) FROM employees;
68. SELECT LENGTH(name) FROM employees;
69. SELECT SUBSTRING(name, 1, 3) FROM employees;
70. SELECT CONCAT(first_name, ' ', last_name) FROM employees;
71. SELECT REPLACE(phone, '-', '') FROM employees;
72. SELECT name FROM employees WHERE name LIKE '%son';
73. SELECT LEFT(name, 1), RIGHT(name, 1) FROM employees;
74. SELECT SPLIT_PART(email, '@', 2) FROM employees; -- get domain from email
75. SELECT POSITION('@' IN email) FROM employees;

```

## Date & Time Functions — #76 to #83

```

76. SELECT CURRENT_DATE, CURRENT_TIMESTAMP;
77. SELECT EXTRACT(YEAR FROM hire_date) FROM employees;
78. SELECT DATE_TRUNC('month', order_date) FROM orders;
79. SELECT hire_date + INTERVAL '30 days' FROM employees;
80. SELECT AGE(CURRENT_DATE, hire_date) FROM employees;
81. SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL '7 days';

```

```

82. SELECT TO_CHAR(order_date, 'YYYY-MM') FROM orders;
83. SELECT EXTRACT(DOW FROM order_date) FROM orders; -- day of week

```

## Constraints, Transactions & Admin — #84 to #100

```

84. ALTER TABLE employees ADD CONSTRAINT pk_emp PRIMARY KEY (id);
85. ALTER TABLE orders ADD CONSTRAINT fk_emp FOREIGN KEY (emp_id) REFERENCES employees(id);
86. ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);
87. ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary > 0);
88. ALTER TABLE employees ALTER COLUMN name SET NOT NULL;
89. BEGIN; -- start a transaction
90. COMMIT; -- save changes permanently
91. ROLLBACK; -- undo changes since BEGIN
92. SAVEPOINT sp1;
93. ROLLBACK TO SAVEPOINT sp1;
94. CREATE VIEW active_employees AS SELECT * FROM employees WHERE active = TRUE;
95. DROP VIEW active_employees;
96. GRANT SELECT ON employees TO analyst_role;
97. REVOKE SELECT ON employees FROM analyst_role;
98. CREATE USER analyst WITH PASSWORD 'secure';
99. EXPLAIN ANALYZE SELECT * FROM employees WHERE salary > 70000; -- query performance plan
100. VACUUM ANALYZE employees; -- reclaim space + refresh stats (Postgres)

```

## Quick Reference

| Need to...                  | Section      |
|-----------------------------|--------------|
| Create/alter a table        | DDL (#1–10)  |
| Insert, update, delete rows | DML (#11–20) |

| Need to...                              | Section                            |
|-----------------------------------------|------------------------------------|
| Filter and sort results                 | Querying (#21–30)                  |
| Combine multiple tables                 | Joins (#31–38)                     |
| Summarize data by group                 | Aggregation (#39–48)               |
| Rank or compare rows without collapsing | Window Functions (#49–58)          |
| Break a query into readable steps       | Subqueries/CTEs (#59–65)           |
| Clean or extract text                   | String Functions (#66–75)          |
| Work with dates                         | Date Functions (#76–83)            |
| Enforce data integrity or manage safely | Constraints/Transactions (#84–100) |

## You're Ready When...

| Section                   | Can you do this without notes?                                    |
|---------------------------|-------------------------------------------------------------------|
| <b>DDL/DML</b>            | Create a table with constraints, then insert/update/delete safely |
| <b>Querying/Joins</b>     | Write any join type and filter with confidence                    |
| <b>Aggregation/Window</b> | Build a groupby summary AND a top-N-per-group window query        |
| <b>Subqueries/CTEs</b>    | Write a recursive CTE for a hierarchy                             |
| <b>Strings/Dates</b>      | Clean text and extract date parts without googling syntax         |
| <b>Transactions/Admin</b> | Explain BEGIN/COMMIT/ROLLBACK and when to use EXPLAIN ANALYZE     |